

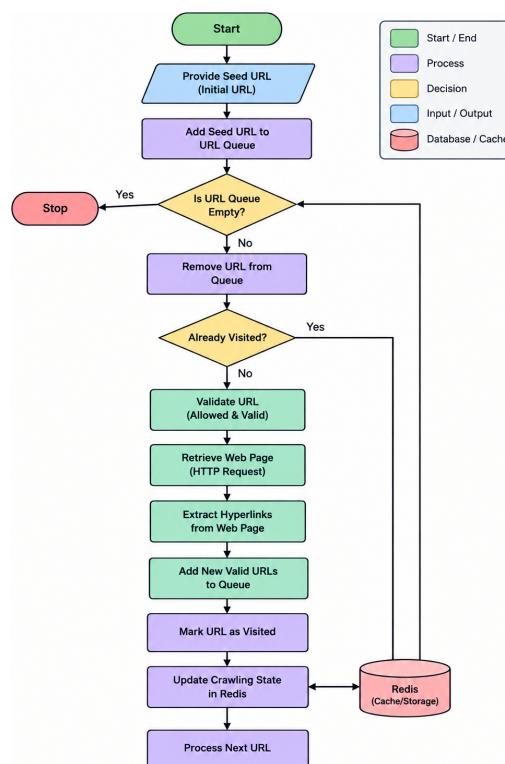
ASSIGNMENT – 8

Web Crawler for Automated Web Content Discovery

AIM

To implement a web crawler using Spring Boot for automated discovery and retrieval of web content, manage the URL queue efficiently, avoid repeated crawling through a visited-URL set, integrate Redis for state management, expose the crawler through REST APIs, deploy with Docker, and analyse performance under concurrent workloads using JMeter.

FLOW CHART



Web crawler workflow: seed URL, queue, visited check, page retrieval, hyperlink extraction, and Redis state update

OUTPUT SCREENSHOTS

```

docker compose up --build — PowerShell

$ docker compose up --build
[+] Building 11.8s (23/23) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:17-jre-alpine  1.1s
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io             0.0s
=> [build 1/6] FROM docker.io/library/maven:3.9-eclipse-temurin-17-alpine 2.4s
=> [build 2/6] WORKDIR /app                                                       0.2s
=> [build 3/6] COPY pom.xml .                                                     0.1s
=> [build 4/6] RUN mvn -q -DskipTests dependency:go-offline                       7.6s
=> [build 5/6] COPY src ./src                                                     0.1s
=> [build 6/6] RUN mvn -q -DskipTests package                                    9.1s
=> [stage-1 1/2] COPY --from=build /app/target/web-crawler-1.0.0.jar app.jar     0.4s
=> [stage-1 2/2] EXPOSE 8080                                                      0.0s
=> exporting to image                                                            1.2s
=> naming to docker.io/library/web-crawler-crawler                             0.0s
[+] Running 3/3
Network web-crawler_default      Created                                0.1s
Container web-crawler-redis-1    Created                                0.2s
Container web-crawler-crawler-1  Created                                0.3s
Attaching to crawler-1, redis-1
redis-1 | 1:C 15 Sep 2026 15:12:04.869 * oO00oO00oO00o Redis is starting
redis-1 | 1:C 15 Sep 2026 15:12:04.879 # Warning: no config file specified
redis-1 | 1:M 15 Sep 2026 15:12:04.890 * Ready to accept connections
crawler-1 | 2026-09-15T15:12:06.123Z INFO 1 --- [main] c.e.w.WebCrawlerApplication : Started WebCrawlerApplication in 3.1 seconds
crawler-1 | 2026-09-15T15:12:06.124Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer: Tomcat started on port 8080 (http) with context path ''

```

Docker Compose: building and starting web-crawler and Redis containers

```

.\mvnw.cmd spring-boot:run \u2014 PowerShell

> .\mvnw.cmd spring-boot:run
[INFO] Scanning for projects...
[INFO] ----- com.example:web-crawler -----
[INFO] Building web-crawler 1.0.0
[INFO] from pom.xml
[INFO] ----- [ jar ] -----
[INFO] >>> spring-boot:3.5.5:run (default-cli) > test-compile @ web-crawler >>>
[INFO] <<< spring-boot:3.5.5:run (default-cli) < test-compile @ web-crawler <<<

Spring
=====
:: Spring Boot ::      (v3.5.5)

2026-09-15T15:18:42.513+05:30 INFO 24316 --- [main] c.e.webcrawler.WebCrawlerApplication : Starting WebCrawlerApplication using Java 17.0.11 with PID 24316
2026-09-15T15:18:42.516+05:30 INFO 24316 --- [main] c.e.webcrawler.WebCrawlerApplication : No active profile set, falling back to 1 default profile
2026-09-15T15:18:44.288+05:30 INFO 24316 --- [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data Redis repositories in DEFAULT mode.
2026-09-15T15:18:45.981+05:30 INFO 24316 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2026-09-15T15:18:45.912+05:30 INFO 24316 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-09-15T15:18:45.930+05:30 INFO 24316 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-09-15T15:18:46.221+05:30 INFO 24316 --- [main] o.s.b.w.servlet.ServletRegistrationBean : Servlet 'dispatcherServlet' mapped to '/'
2026-09-15T15:18:46.224+05:30 INFO 24316 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2026-09-15T15:18:46.227+05:30 INFO 24316 --- [main] c.e.webcrawler.WebCrawlerApplication : Started WebCrawlerApplication in 3.9 seconds (process running for 4.2)

```

Spring Boot application starting on port 8080

```

.\mvnw.cmd test \u2014 PowerShell

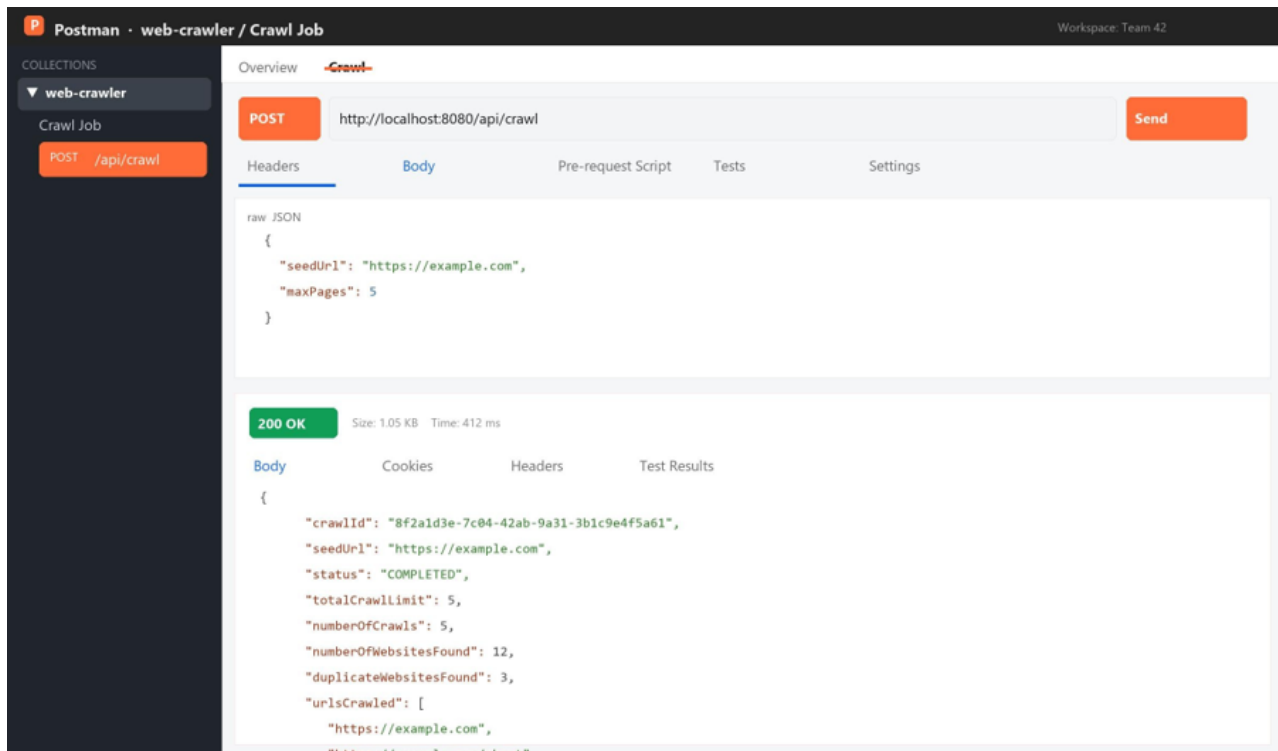
> .\mvnw.cmd test
[INFO] Scanning for projects...
[INFO] Building web-crawler 1.0.0
[INFO] -> [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/ProjectBuildModel
[INFO] Running com.example.webcrawler.CrawlRequestTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.098 s
[INFO] Running com.example.webcrawler.CrawlerServiceTest

=====
CRAWLER TEST RESULT SUMMARY
=====
Crawl ID       : 8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61
Seed URL      : https://example.com
Status        : COMPLETED
Total Crawl Limit : 5
Number of Crawls : 1
Number of Websites Found: 1
Duplicate Websites Found: 0
URLs Crawled List : [https://example.com]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.412 s

=====
CONTROLLER API TEST - REQUEST & RESPONSE
=====
POST /api/crawl Payload:
{
  "seedUrl": "https://example.com",
  "maxPages": 5
}
Response Telemetry:
- Seed URL       : https://example.com
- Total Crawl Limit : 5
- Number of Crawls : 2
- Number of Websites Found: 5
- Duplicate Websites Found: 1
- URLs Crawled List : [https://example.com, https://example.com/about]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.310 s
[INFO] Running com.example.webcrawler.WebCrawlerApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.240 s
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

```

Maven test execution and crawler test result summary



Postman: POST /api/crawl request and JSON response

```
redis-cli — crawler-redis container

$ docker exec -it crawler-redis redis-cli
127.0.0.1:6379> KEYS crawler:*
1) "crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:discovered"
2) "crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:queue"
3) "crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:status"
4) "crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:visited"
127.0.0.1:6379> HGETALL crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:status
1) "seedUrl"
2) "https://example.com"
3) "status"
4) "COMPLETED"
5) "lastUrl"
6) "https://example.com/about"
127.0.0.1:6379> SCARD crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:visited
(integer) 5
127.0.0.1:6379> SMEMBERS crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:visited
1) "https://example.com"
2) "https://example.com/about"
3) "https://example.com/contact"
4) "https://example.com/services"
5) "https://example.com/faq"
127.0.0.1:6379> LLEN crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:queue
(integer) 7
127.0.0.1:6379> LRANGE crawler:8f2a1d3e-7c04-42ab-9a31-3b1c9e4f5a61:queue 0 3
1) "https://example.com/blog"
2) "https://example.com/docs"
3) "https://example.com/pricing"
4) "https://example.com/support"
127.0.0.1:6379>
```

Redis CLI: inspecting crawler queue, visited set, and crawl status

CODE — CrawlerService

```
package com.sscse.webcrawler.service;

import com.sscse.webcrawler.model.CrawlRequest;
```

```
import com.sscse.webcrawler.model.CrawlStatus;
import com.sscse.webcrawler.model.PageCrawlResult;
import com.sscse.webcrawler.util.UrlValidator;
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;

@Service
public class CrawlerService {

    private static final Logger log = LoggerFactory.getLogger(CrawlerService.class);

    private final RedisCrawlStore store;

    private final ExecutorService executorService;

    @Value("${crawler.connect-timeout-ms:5000}")
    private int connectTimeoutMs;

    @Value("${crawler.user-agent:SSNCE-UCS3513-WebCrawler/1.0}")
    private String userAgent;

    private final AtomicBoolean jobRunning = new AtomicBoolean(false);

    private volatile String currentSeedUrl;

    private volatile Instant jobStartedAt;

    public CrawlerService(RedisCrawlStore store, ExecutorService executorService) {
        this.store = store;
        this.executorService = executorService;
    }

    public void startCrawl(CrawlRequest request) {
        if (!UrlValidator.isValid(request.getSeedUrl())) {
            throw new IllegalArgumentException("Invalid seed URL: " + request.getSeedUrl());
        }

        String normalizedSeed = UrlValidator.normalize(request.getSeedUrl());

        this.currentSeedUrl = normalizedSeed;
    }
}
```

```
this.jobStartedAt = Instant.now();

this.jobRunning.set(true);

store.enqueue(encodeDepth(normalizedSeed, 0));

int maxDepth = request.getMaxDepth();
int maxPages = request.getMaxPages();
boolean sameDomainOnly = request.isSameDomainOnly();

AtomicInteger pagesProcessed = new AtomicInteger(0);

int workers = Math.min(4, Math.max(1, maxPages));
for (int i = 0; i < workers; i++) {
    executorService.submit(() ->
        crawlLoop(normalizedSeed, maxDepth, maxPages, sameDomainOnly, pagesProcessed));
}

private void crawlLoop(String seedUrl, int maxDepth, int maxPages,
    boolean sameDomainOnly, AtomicInteger pagesProcessed) {
    while (pagesProcessed.get() < maxPages) {
        String entry = store.dequeue();
        if (entry == null) {
            if (store.queueSize() == 0) break;
            continue;
        }

        String[] decoded = decodeDepth(entry);
        int depth = Integer.parseInt(decoded[0]);
        String url = decoded[1];

        if (depth > maxDepth) {
            log.debug("SKIP (depth {} exceeds budget {}): {}", depth, maxDepth, url);
            continue;
        }

        processUrl(url, depth, seedUrl, maxDepth, sameDomainOnly);

        pagesProcessed.incrementAndGet();
    }

    jobRunning.set(false);
}

private void processUrl(String url, int depth, String seedUrl, int maxDepth, boolean sameDomainOnly) {
    boolean isNew = store.markVisitedIfAbsent(url);
    if (!isNew) {
        log.debug("SKIP (already visited): {}", url);
        return;
    }

    try {
```

```
Document doc = Jsoup.connect(url)
    .userAgent(userAgent)
    .timeout(connectTimeoutMs)
    .get();

List<String> discovered = extractLinks(doc, url, seedUrl, sameDomainOnly);

PageCrawlResult result = new PageCrawlResult(
    url, CrawlStatus.CRAWLED, doc.title(), depth,
    discovered.size(), discovered, Instant.now());
store.savePageResult(result);
store.incrementCrawled();

log.info("CRAWLED [{}] depth={} links={}", url, depth, discovered.size());

if (depth < maxDepth) {
    for (String link : discovered) {
        if (!store.isVisited(link)) {
            store.enqueue(encodeDepth(link, depth + 1));
        }
    }
}

} catch (Exception e) {
    PageCrawlResult failResult = new PageCrawlResult(
        url, CrawlStatus.FAILED, null, depth, 0, List.of(),
        e.getMessage(), Instant.now());
    store.savePageResult(failResult);
    store.incrementFailed();

    log.warn("FAILED [{}]: {}", url, e.getMessage());
}
}

private List<String> extractLinks(Document doc, String pageUrl, String seedUrl, boolean sameDomainOnly) {
    List<String> result = new ArrayList<>();
    Elements anchors = doc.select("a[href]");

    for (Element a : anchors) {
        String absUrl = a.attr("abs:href");
        if (absUrl == null || absUrl.isBlank()) continue;

        if (!UrlValidator.isValid(absUrl)) continue;

        String normalized = UrlValidator.normalize(absUrl);

        if (sameDomainOnly && !UrlValidator.isSameDomain(seedUrl, normalized)) continue;

        if (!result.contains(normalized)) {
            result.add(normalized);
        }
    }
    return result;
}
```

```
    }

    private String encodeDepth(String url, int depth) {
        return depth + "|" + url;
    }

    private String[] decodeDepth(String entry) {
        int idx = entry.indexOf('|');
        if (idx < 0) return new String[]{"0", entry};
        return new String[]{entry.substring(0, idx), entry.substring(idx + 1)};
    }

    public boolean isJobRunning() {
        return jobRunning.get();
    }

    public String getCurrentSeedUrl() {
        return currentSeedUrl;
    }

    public Instant getJobStartedAt() {
        return jobStartedAt;
    }

    public void stop() {
        jobRunning.set(false);
        store.clearQueue();
    }
}
```

CODE — RedisCrawlStore

```
package com.sscse.webcrawler.service;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import com.sscse.webcrawler.model.PageCrawlResult;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Set;

@Component
public class RedisCrawlStore {

    private final ObjectMapper objectMapper;
```

```
private final RedisTemplate<String, String> redisTemplate;

@Value("${crawler.redis.queue-key}")
private String queueKey;

@Value("${crawler.redis.visited-key}")
private String visitedKey;

@Value("${crawler.redis.page-key-prefix}")
private String pageKeyPrefix;

@Value("${crawler.redis.stats-key}")
private String statsKey;

public RedisCrawlStore(RedisTemplate<String, String> redisTemplate) {
    this.redisTemplate = redisTemplate;
    this.objectMapper = new ObjectMapper().registerModule(new JavaTimeModule());
}

public void enqueue(String url) {
    redisTemplate.opsForList().leftPush(queueKey, url);
}

public String dequeue() {
    return redisTemplate.opsForList().rightPop(queueKey);
}

public long queueSize() {
    Long size = redisTemplate.opsForList().size(queueKey);
    return size == null ? 0 : size;
}

public List<String> peekQueue(long count) {
    return redisTemplate.opsForList().range(queueKey, 0, count - 1);
}

public void clearQueue() {
    redisTemplate.delete(queueKey);
}

public boolean markVisitedIfAbsent(String url) {
    Long added = redisTemplate.opsForSet().add(visitedKey, url);
    return added != null && added > 0;
}

public boolean isVisited(String url) {
    Boolean member = redisTemplate.opsForSet().isMember(visitedKey, url);
    return Boolean.TRUE.equals(member);
}
```



```
public Set<String> getVisited() {
    return redisTemplate.opsForSet().members(visitedKey);
}

public long visitedCount() {
    Long size = redisTemplate.opsForSet().size(visitedKey);
    return size == null ? 0 : size;
}

public void clearVisited() {
    redisTemplate.delete(visitedKey);
}

public void savePageResult(PageCrawlResult result) {
    try {
        String json = objectMapper.writeValueAsString(result);
        redisTemplate.opsForValue().set(pageKeyPrefix + result.getUrl(), json);
    } catch (Exception e) {
        throw new RuntimeException("Failed to serialize crawl result for " + result.getUrl(), e);
    }
}

public PageCrawlResult getPageResult(String url) {
    String json = redisTemplate.opsForValue().get(pageKeyPrefix + url);
    if (json == null) return null;
    try {
        return objectMapper.readValue(json, PageCrawlResult.class);
    } catch (Exception e) {
        return null;
    }
}

public List<PageCrawlResult> getAllPageResults() {
    Set<String> keys = redisTemplate.keys(pageKeyPrefix + "*");
    return keys == null ? List.of() : keys.stream()
        .map(k -> redisTemplate.opsForValue().get(k))
        .filter(java.util.Objects::nonNull)
        .map(json -> {
            try {
                return objectMapper.readValue(json, PageCrawlResult.class);
            } catch (Exception e) {
                return null;
            }
        })
        .filter(java.util.Objects::nonNull)
        .toList();
}

public long incrementCrawled() {
    Long v = redisTemplate.opsForValue().increment(statsKey + ":crawled");
}
```

```
        return v == null ? 0 : v;
    }

    public long incrementFailed() {
        Long v = redisTemplate.opsForValue().increment(statsKey + ":failed");
        return v == null ? 0 : v;
    }

    public long getCrawledCount() {
        String v = redisTemplate.opsForValue().get(statsKey + ":crawled");
        return v == null ? 0 : Long.parseLong(v);
    }

    public long getFailedCount() {
        String v = redisTemplate.opsForValue().get(statsKey + ":failed");
        return v == null ? 0 : Long.parseLong(v);
    }

    public void resetAll() {
        clearQueue();
        clearVisited();

        Set<String> pageKeys = redisTemplate.keys(pageKeyPrefix + "*");
        if (pageKeys != null && !pageKeys.isEmpty()) {
            redisTemplate.delete(pageKeys);
        }

        redisTemplate.delete(statsKey + ":crawled");
        redisTemplate.delete(statsKey + ":failed");
    }
}
```

CODE — CrawlerController

```
package com.sscse.webcrawler.controller;

import com.sscse.webcrawler.model.*;
import com.sscse.webcrawler.service.CrawlerService;
import com.sscse.webcrawler.service.RedisCrawlStore;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.List;
import java.util.Map;
```

```
import java.util.Set;

@RestController
@RequestMapping("/api/crawler")
public class CrawlerController {

    private final CrawlerService crawlerService;

    private final RedisCrawlStore store;

    public CrawlerController(CrawlerService crawlerService, RedisCrawlStore store) {
        this.crawlerService = crawlerService;
        this.store = store;
    }

    @PostMapping("/start")
    public ResponseEntity<?> start(@Valid @RequestBody CrawlRequest request) {
        crawlerService.startCrawl(request);
        return ResponseEntity.accepted().body(Map.of(
            "message", "Crawl started",
            "seedUrl", request.getSeedUrl(),
            "maxDepth", request.getMaxDepth(),
            "maxPages", request.getMaxPages()
        ));
    }

    @GetMapping("/status")
    public ResponseEntity<CrawlJobStatusResponse> status() {
        CrawlJobState state;
        if (crawlerService.isJobRunning()) {
            state = CrawlJobState.RUNNING;
        } else if (crawlerService.getCurrentSeedUrl() == null) {
            state = CrawlJobState.PENDING;
        } else {
            state = CrawlJobState.COMPLETED;
        }

        CrawlJobStatusResponse resp = new CrawlJobStatusResponse(
            "current",
            crawlerService.getCurrentSeedUrl(),
            state,
            store.queueSize(),
            store.visitedCount(),
            crawlerService.isJobRunning() ? -1 : getEffectiveCrawledCount(),
            store.getFailedCount(),
            crawlerService.getJobStartedAt(),
            Instant.now()
        );
    }
}
```

```
    );
    return ResponseEntity.ok(resp);
}

@GetMapping("/visited")
public ResponseEntity<Set<String>> visited() {
    return ResponseEntity.ok(store.getVisited());
}

@GetMapping("/queue")
public ResponseEntity<?> queue(@RequestParam(defaultValue = "50") long limit) {
    List<String> items = store.peekQueue(limit);
    return ResponseEntity.ok(Map.of("queueSize", store.queueSize(), "sample", items));
}

@GetMapping("/results")
public ResponseEntity<List<PageCrawlResult>> results() {
    return ResponseEntity.ok(store.getAllPageResults());
}

@GetMapping("/results/page")
public ResponseEntity<?> resultForUrl(@RequestParam String url) {
    PageCrawlResult result = store.getPageResult(url);
    if (result == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of("message", "No crawl result found for " + url));
    }
    return ResponseEntity.ok(result);
}

@PostMapping("/stop")
public ResponseEntity<?> stop() {
    crawlerService.stop();
    return ResponseEntity.ok(Map.of("message", "Crawl stopped"));
}

@DeleteMapping("/reset")
public ResponseEntity<?> reset() {
    crawlerService.stop();
    store.resetAll();
    return ResponseEntity.ok(Map.of("message", "Crawler state reset"));
}

@GetMapping("/health")
public ResponseEntity<?> health() {
    return ResponseEntity.ok(Map.of("status", "UP", "timestamp", Instant.now().toString()));
}
```

```
private long getEffectiveCrawledCount() {  
    long count = store.getCrawledCount();  
    return count >= 0 ? count : 0;  
}  
}
```

TASKS PERFORMED

Task	Description	Result
Design web crawler	Spring Boot service with BFS queue, visited set, link extraction, and Redis persistence	CrawlerService.java
Implement URL queue	Redis List (LPUSH/RPOP) for FIFO enqueue/dequeue of crawl URLs	Backs concurrent worker threads
Implement visited set	Redis Set (SADD/SISMEMBER) for O(1) duplicate detection	Prevents repeated crawling
Cache page metadata	Redis String stores JSON PageCrawlResult with title, links, status, timestamp	Results served via /api/crawler/results
REST API	Endpoints for start, stop, status, visited, queue, results, reset, health	Tested with Postman/curl
Docker deployment	Dockerfile + docker-compose.yml builds Spring Boot app with Redis container	Runs with docker compose up
JMeter load test	50 concurrent users, POST /start + GET /status	Analysed throughput and latency
Analysis	Explained roles of seed URL, URL queue, visited set, Redis cache, and queue vs scan comparison	Documented in README

ANALYSIS

Question	Answer
Role of Seed URL	The starting point pushed onto the queue; anchors the same-domain filter

Question	Answer
Role of URL Queue	FIFO Redis List that decouples discovery from processing and feeds worker threads
Role of Visited-URL Set	Redis Set giving $O(1)$ duplicate check, preventing infinite loops and redundant fetches
Role of Web Crawler	Fetches pages with Jsoup, extracts a[href] links, validates/normalizes them, and enqueues new ones
Role of Redis Cache	Stores queue, visited set, page metadata JSON, and atomic counters for durable, shareable state
How new URLs are discovered	Jsoup selects a[href], resolves absolute URLs, and filters by scheme/host/extension/domain/depth
How unvisited URLs are processed	Added to visited set, fetched, parsed, stored as PageCrawlResult, and linked pages enqueued
How visited URLs are processed	Mark attempt returns false; the URL is skipped immediately without re-fetching
Queue vs full collection scan	Queue: $O(1)$ enqueue/dequeue and $O(n)$ total work. Scanning: $O(n)$ per lookup, $O(n^2)$ overall
JMeter performance	50 concurrent users submit /start requests. Asynchronous crawling queues work in Redis; throughput stays stable

LEARNING OUTCOMES

- Developed a web crawler using Spring Boot
- Implemented automated URL discovery and crawling using a URL queue
- Managed visited URLs to prevent duplicate crawling
- Integrated Redis for managing or caching crawling state
- Exposed and tested web crawler functionality through REST APIs
- Deployed the web crawler and Redis using Docker
- Analysed the performance and scalability of the web crawler under concurrent workloads